

CS 551 Systems Programming

Lecture 0: Getting Started: Rust and Cargo

Why Rust

What we're doing here, and why in this language

What Rust is

- **A systems programming language.** You have full access to memory management. There is no garbage collector. You can get direct access to hardware.
- **Statically typed, with a very strict compiler.** Most of what would be a runtime bug in another language is a compile error here.
- **Multi-paradigm.** Functional features, generics, first-class concurrency.

The headline: it is genuinely hard to write memory-unsafe code in Rust without saying the word `unsafe` first.

Why not C or C++

C and C++ carry a lot of technical baggage. C is from 1972.

- A large chunk of a course like this, taught in C, becomes weird C trivia: integer promotion rules, undefined behavior, what your particular compiler does on Tuesdays. That is not what I want to teach you.
- There is no real build system. There is no dependency management. `make` is not a build system, it is a scripting language that runs `cc`.
- You carry a lot in your head, all the time, and the compiler does not help you.

Rust gives you the same machine access with a compiler that actually participates.

The second reason, and the one this course is built on

Rust makes **checkable claims** about performance. It compiles ahead of time, with no JIT and no garbage collector, so what you measure is what shipped.

That makes it a place where you can *run the experiment* rather than repeat what you were told.

We will do exactly that in Lectures 6, 13, and 21.

Rust's downsides, honestly

- **C is a very simple language.** You can hold all of C in your head. Rust is very much not simple, and you cannot.
- **The learning curve is real.** You will fight the compiler for a few weeks. This is normal and it does end.
- **C++ is a nightmare in general**, so it is not much of a defense of C++, but it is fair to say Rust has real complexity of its own.

The deal: you pay upfront in learning curve and compile time. You get back whole categories of bugs you cannot ship.

The toolchain

`rustup`, `cargo`, and the docs

Installing: `rustup`

`rustup` is the toolchain installer and version manager. It is the only supported way to install Rust.

```
1 curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

- Getting started: <https://www.rust-lang.org/learn/get-started>
- `rustup` book: <https://rust-lang.github.io/rustup/>

We use the current stable toolchain. To pin a specific version:

```
1 rustup toolchain install 1.97.0
2 rustup default 1.97.0
```

Before you install anything: the playground

<https://play.rust-lang.org>

Runs real `rustc` in the browser. Good for:

- Trying a snippet from the slides without a project.
- Sharing a broken program with me or with a classmate. It generates a permalink.
- Checking what a different edition or release channel does with the same code.

It is not a substitute for a local toolchain. You need the local one for the projects.

Rust has modern tooling

Modern languages ship a real build and dependency toolchain. Rust's is called **cargo**, and it is one of the main reasons to pick this language.

One tool handles:

- Building and running your project.
- Fetching, versioning, and building dependencies.
- Running tests and benchmarks.
- Generating documentation.
- Formatting and linting.

You will use `cargo` for everything in this course.

cargo new <project-name>

Creates a new project. By default:

- The project goes in a directory named after the project.
- It is set up as a **binary** (an application), not a library. Use `--lib` for a library.
- It is set up as a git repository.

```
1 cargo new --bin guessing_game
2 cd guessing_game
```

Cargo.toml

The project's configuration: metadata plus dependencies.

```
1 [package]
2 name = "guessing_game"
3 version = "0.1.0"
4 edition = "2024"
5
6 [dependencies]
```

Editions are Rust's mechanism for making breaking language changes without breaking old code. A 2015-edition crate and a 2024-edition crate interoperate fine. We use **2024** throughout this course.

We come back to this file when we add a dependency.

cargo run, cargo build, cargo check

- `cargo run`. Builds the project and executes it. What you use while developing.
- `cargo build`. Builds it. Does not run it. Fetches and builds dependencies too.
- `cargo check`. Type-checks and borrow-checks without generating code. Substantially faster than a full build.

`cargo check` is the one to run in a loop while you are fighting the compiler. You do not care about the binary yet; you care about the errors.

Add `--release` to `build` or `run` for an optimized build. Debug builds are slow on purpose.

Hello, world

```
1 fn main() {  
2     println!("hello world!");  
3 }
```

Hello, world

```
1 fn main() {  
2     println!  
3 }
```

fn declares a function. **main** is the entry point, same as C

Hello, world

```
1 fn main() {  
2     println!  
3 }
```

Braces enclose a block, same as C. This one runs from line 1 to line 3

Hello, world

```
1 fn main() {  
2     println!("hello world!");  
3 }
```

`println!` is a **macro**, not a function. The `!` is how you tell

Hello, world

```
1 fn main() {  
2     println!("hello world!");  
3 }
```

Statements end with a semicolon



A macro is not a function!

Three of the four things on this slide come straight from C: the function, the braces, the semicolon. The `!` is the one you could not have guessed, and it is the one that comes back.

You could use `rustc` directly

```
1 rustc main.rs    # produces an executable named `main`  
2 ./main
```

This works, and it is worth doing exactly once so you know `cargo` is not magic.

You will not do this again. 99.99% of the time you invoke `cargo`, and `cargo` invokes `rustc` for you with the right flags, the right dependency paths, and the right edition.

Your editor: VS Code and rust-analyzer

You can write Rust in any editor. This course assumes **VS Code**, because it is free, cross-platform, and the setup is two steps.

1. Install VS Code from <https://code.visualstudio.com/>.
2. Open the Extensions panel (`Shift-Cmd-X` on macOS, `Ctrl-Shift-X` elsewhere), search for **rust-analyzer**, and install the one published by `rust-lang`.

There is no third step. `rust-analyzer` finds the toolchain `rustup` installed for you and reads your `Cargo.toml` to learn what the project depends on.

Do not install the old extension called `Rust`. It is deprecated, it is superseded by `rust-analyzer`, and running both makes each of them worse.

rust-analyzer is the compiler, in your editor

It is not a syntax highlighter with opinions. It runs real `cargo check` analysis in the background, so what it underlines is what `rustc` would say.

- **Inline type hints.** `let mut guess = String::new();` shows you the `: String` it inferred, in gray. The fastest way to see what inference decided.
- **Errors where you wrote them.** The `E0277` from a few slides back appears under the offending expression as you type, not after you switch to a terminal.
- **Go to definition** (`F12`) into your code *and* into the standard library.
- **Hover for docs.** `read_line`'s documentation without leaving the file.
- **Quick fixes.** "Fill match arms" writes out every variant; "Import `std::io`" adds the `use` line.

Two settings worth turning on

Format on save. Set `"editor.formatOnSave": true` and rustfmt runs on every save. You will never again lose points to formatting.

Clippy instead of check. Set `rust-analyzer.check.command` to `"clippy"` and the editor underlines lint warnings, not just errors.

```
1 // settings.json
2 {
3     "editor.formatOnSave": true,
4     "rust-analyzer.check.command": "clippy"
5 }
```

Open the folder, not the file. rust-analyzer needs to see `Cargo.toml`. Open the project directory; a lone `.rs` file gives it nothing to work with.

Set up your environment now

Two more tools. `rustup` already installed both:

- `clippy` (`cargo clippy`). The linter. It catches non-idiomatic code, not just errors. The projects grade on a clean clippy run.
- `rustfmt` (`cargo fmt`). The formatter. The projects grade on this too: if `cargo fmt` changes your repository, you lose points.

Run `cargo fmt && cargo clippy` before every commit. Make it a habit now, while the stakes are a homework-sized program.

Recap

1. **Rust gives you C's machine access with a compiler that participates.** You pay in learning curve and compile time.
2. **rustup manages toolchains**, and is the only supported way to install Rust.
3. **The playground** runs real **rustc** in a browser, and is not a substitute for a local toolchain.
4. **cargo is the whole toolchain**: build, run, test, document, format, lint, and dependencies. You will rarely invoke **rustc** yourself.
5. **cargo check is the one to run in a loop** while you are fighting the compiler. It skips code generation and is much faster.
6. **Editions** let the language make breaking changes without breaking old code. We use 2024.
7. **VS Code plus rust-analyzer.** Inline types, errors as you type, format on save.

Exercises

1. Install `rustup` and confirm `rustc --version` and `cargo --version` both work.
2. `cargo new --bin hello`, then run it three ways: `cargo run`, `cargo build` followed by running the binary directly, and `rustc src/main.rs` followed by `./main`. Note which produced a `target/` directory.
3. Open the same project in VS Code with rust-analyzer installed. Introduce a type error and watch where it appears before you save.
4. Run `cargo check` and `cargo build` on the same clean project and time both. Explain the difference.
5. Open <https://play.rust-lang.org>, paste your `main.rs`, and generate a permalink. That link is how you ask for help in this course.

Reading

- **The Rust Programming Language**, chapter 1 (Getting Started). Free at <https://doc.rust-lang.org/book/>.
- **The Cargo Book**, the opening chapters: <https://doc.rust-lang.org/cargo/>
- **The rustup book**, if you ever need more than one toolchain: <https://rust-lang.github.io/rustup/>

Course supplements, in `supplements/`:

- `modules-and-workspaces.md`. Gigafizz hands you a workspace and, from rung 5, asks you to add modules. Read this before you get there.
- `testing.md`. The projects require a clean `cargo test`.

Next: Lecture 1

Your First Program

The guessing game, built line by line, and the five ideas it drags in with it.